

SYNTACTIC STRUCTURES

Constituency Grammars (1/3)

Constituency - the fundamental idea of constituency is that **group of words** may behave as a **single unite** or **phrase**, called constituent.

NP: noun phrase

Single word: *She, Michael*

Phrase: *the house, Russian Hill*

Constituency Grammars (2/3)

- Why not stay with sequence models?
 - Lots of linguistics research has established that Markov chains can't be “what language is made of”
- Field linguistics, language preservation
 - Grammars make it possible to write down key aspects of what is known about a language

Constituency Grammars (3/3)

- Psycholinguistics
 - Language deficits can involve the grammar
 - Relationship between language and cognition
 - Language acquisition
- Practical purposes
 - Language modeling + better estimates, better contexts
 - Disambiguation
 - Example: searching on *voice over IP companies and conferences*.
 - We don't want "[voice] over [IP companies]", we want "[voice over IP] companies"
 - Driver for semantics. **"Who did what to whom?"**

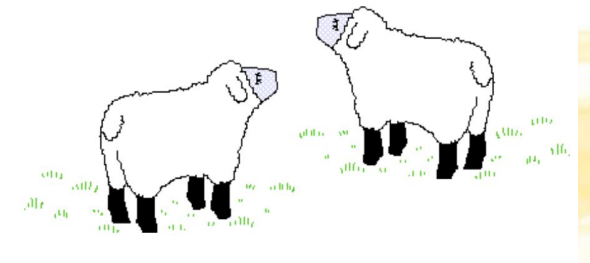
Formal Language

A formal language L over an alphabet Σ is a set of words (strings) over that alphabet.

- $L = \{w_1, w_2, w_3, \dots\}$
- $\Sigma = \{s_1, s_2, s_3, \dots\}$

For example, consider sheep-talk:

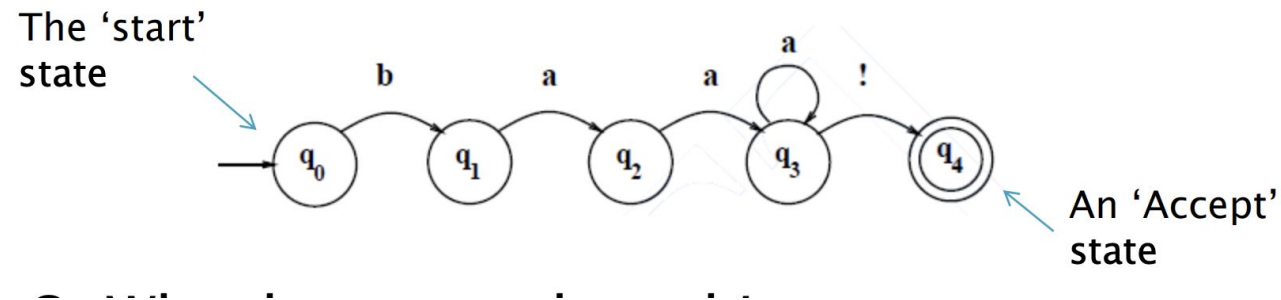
- $L = \{\text{"baa!"}, \text{"baaa!"}, \text{"baaaa!"}, \text{"baaaaa!"} \dots\}$
- $\Sigma = \{\text{'b'}, \text{'a'}, \text{'!'}\}$



Finite-state Automata

A finite-state machine, finite-state automaton (plural: automata) is a mathematical model of computation. It is an abstract machine that can be in one of the finite states at any given time.

Visually, finite state automata are drawn as graphs with nodes that stand for the states and links that stand for the transitions per input. For example:

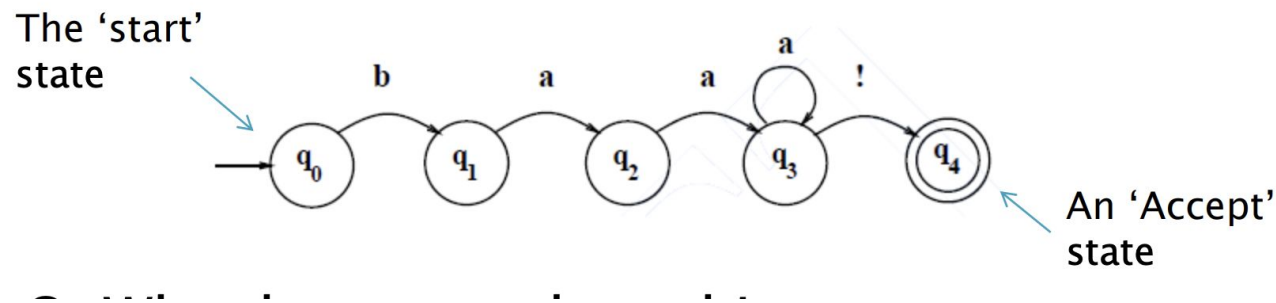


Finite-state Automata

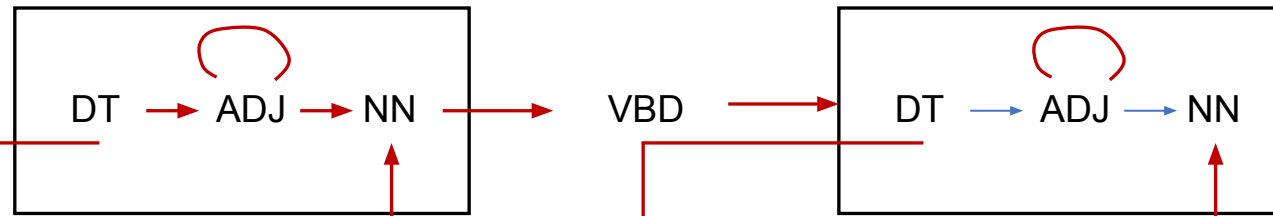
For example, the FSA below is defined as follows:

- $Q = \{q_0, q_1, q_2, q_3, q_4\}$
- $\Sigma = \{'a', 'b', '!\'}$
- q_0 - the start state
- $F = q_4$
- $(q, i) =$

	Input		
State	b	a	!
0	1	0	0
1	0	2	0
2	0	3	0
3	0	3	4
4:	0	0	0



Language Isn't Finite State

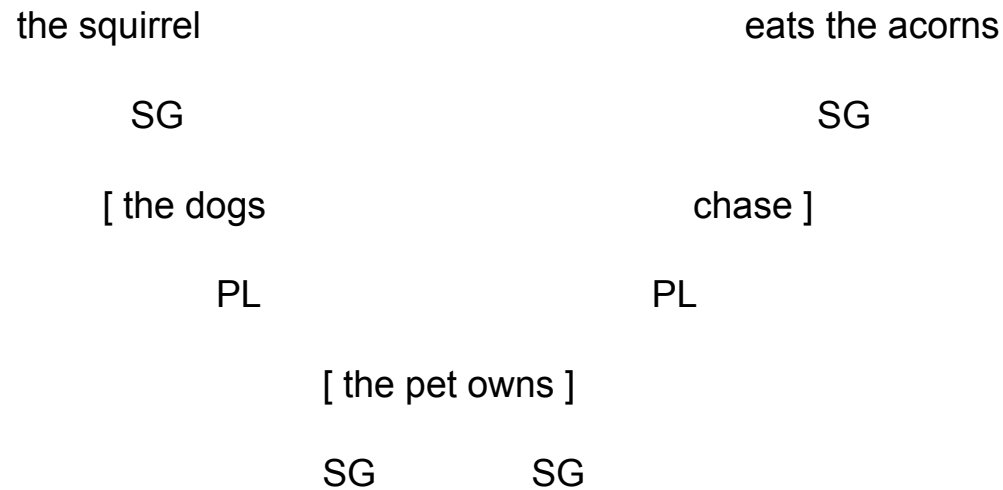


(showing high probability arcs only)

- The fast fast dog chased the terrified squirrel
- The terrified squirrel was chased by the fast fast dog
- Suggests the idea of syntactic constituents being units of grammatical behavior

Center-embedded Dependencies

- Deeper problem with sequence models: center-embedded dependencies

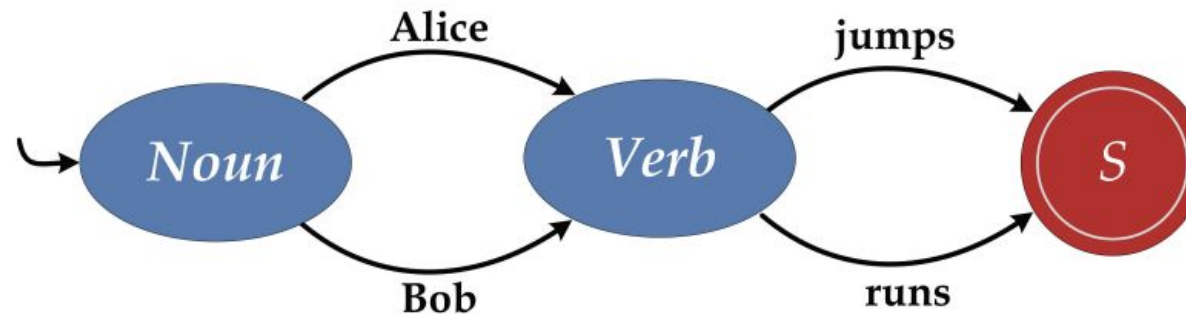


- Consider even just the requirement that the number of corresponding symbols should match
- No finite-state device can generate exactly this language because they are memoryless

$$\{a^n c b^n\}$$

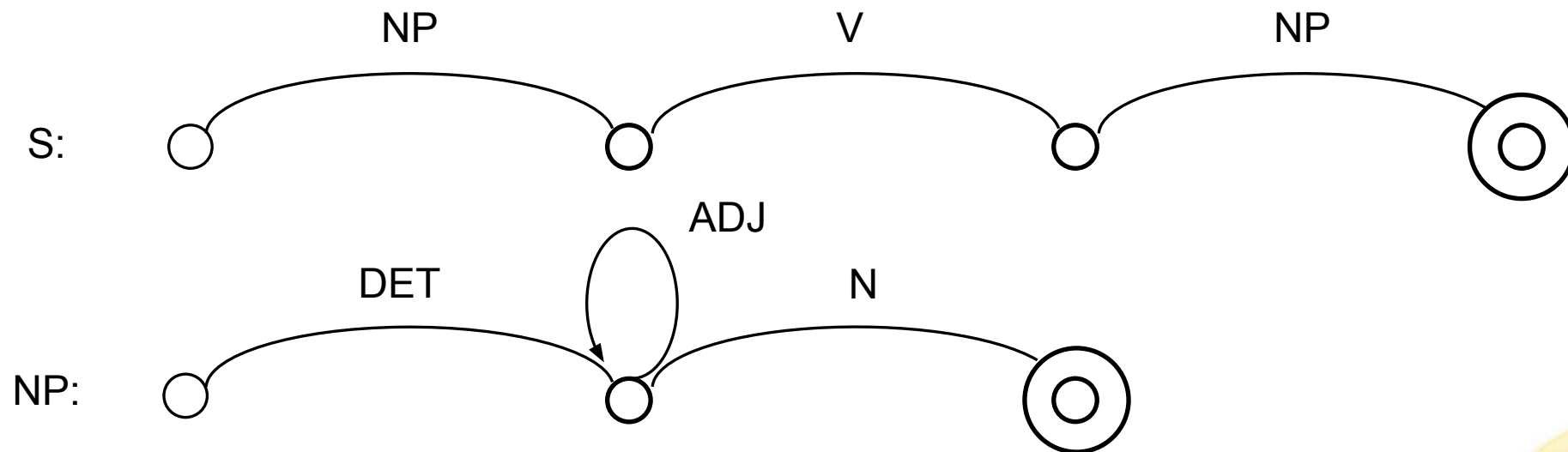
Recursive Transitive Network

A recursive transition network (RTN) - is defined by a graph of nodes and edges. The edges are labeled with output symbols—these are the primitives in the language. The nodes and edge structure provides the means of combination.



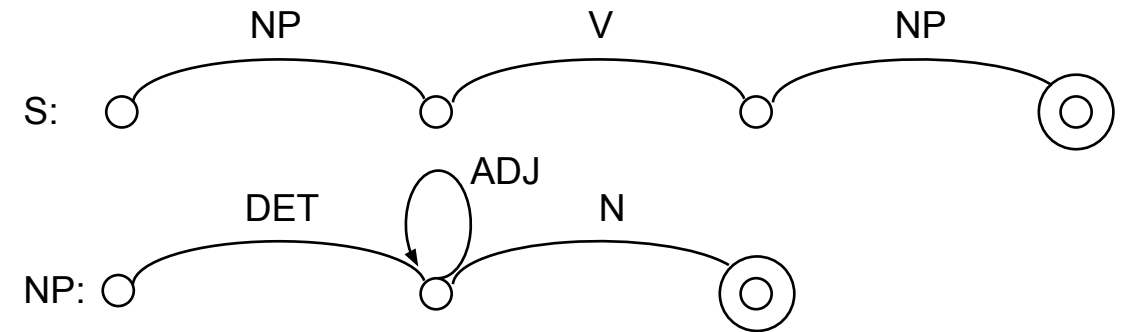
Properties of Constituents (1/2)

This is an example of a **recursive transition network**:



Properties of Constituents

- The constituent NP can now be referred to anywhere, as long as its **internal structure** can be expected to be the **same anywhere**
- Constituents appear in **similar environments**
- Constituents **behave as a unit**, e.g. movement of words affects the whole thing, not individual words
- Constituents often capture a semantic unit (which is often associated with the “head”)



Context-Free Grammars

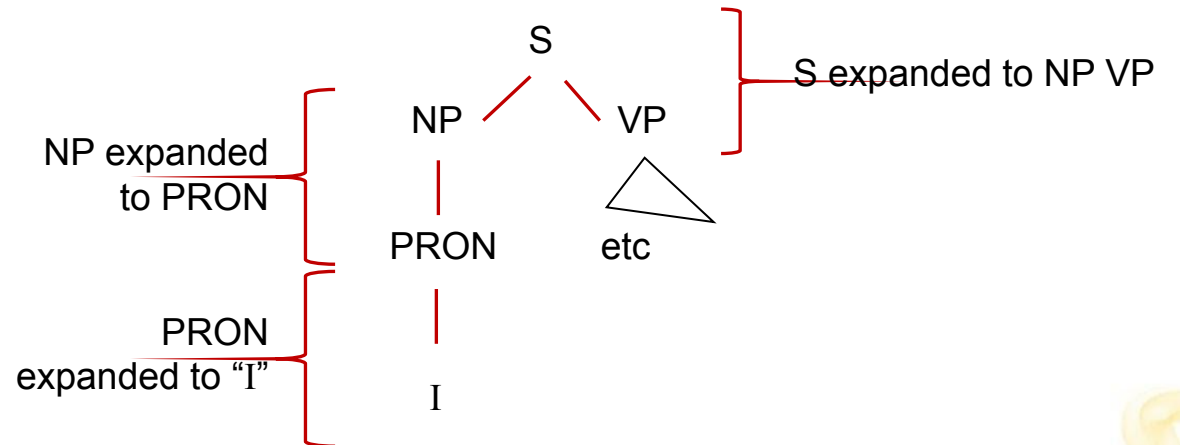
The most commonly used *mathematical system* for modeling constituent structure in English and other natural languages is the **Context-Free Grammar**, or **CFG**. Context-Free Grammars are also called **Phrase-Structure Grammars**.

Formal Definition of Context-Free Grammar

- Context-Free (Phrase Structure) Grammars are a **rewriting system** that form a generative model (4-tuple):
 - Σ : set of **terminal symbols** that are observable (disjoint from N)
 - N : set of **non-terminal symbols** (or **variables**)
 - R : set of **rules** of production (context-free rewriting rules):
 - each rule have the form $A \rightarrow B$; s.t. A is a non-terminal, B is a string of symbols from the finite set of string $(\Sigma \cup N)^*$
 - S : designated **start symbol**

Rules

- $A \rightarrow (N \cup \Sigma)^*$
- Derivation as rewriting a string
- Derivations can also be viewed as series of *events*
- A “**derivation tree**” is a history of the events in the derivation
- Derivation tree encodes a structure for the sentence



Subcategorization

Subcategorization and dependency relation - refers to certain kinds of relations between **words** and **phrases**.

Subcategorization (1/2)

- S ☐ NP VP
- VP ☐ V NP
- VP ☐ V
- NP ☐ Det N
- N ☐ student
- N ☐ students
- V ☐ loved
- V ☐ laughed

Complement of the verb



The student loved the book.

The student laughed the book. Uh oh...

Subcategorization (2/2)

S □ NP VP

VP □ V NP

VP □ V

NP □ Det N

N □ student

N □ students

V □ loved

V □ laughed

Complement
of the verb

The student loved the book.

The student laughed the book.

- Can solve the problem by distinguishing **subcategories** of the V category
- Formally, just some new nonterminals...

V_T □ loved | wrote | Transitive

V_I □ laughed | slept | Intransitive

a) VP □ V_I laughed

b) VP □ V_T NP loved

c) VP □ V_{DIT} NP NP } gave gave Bill the book

d) VP □ V_{DT} NP to NP } gave gave the book to Bill

subcategorization frames



Agreement

- **OK:** The student (SG) loves (SG) the book
- **Not ok:** The students (PL) loves (SG) the book

Similar issue for gender agreement in languages that have it: *el (masc) libro (masc)*, not *la (fem) libro (masc)*

Solution: expand the grammar with multiple sets of rules:

PLNoun -> students | flights |

...

SGNoun -> student | flight | ...

Problems:

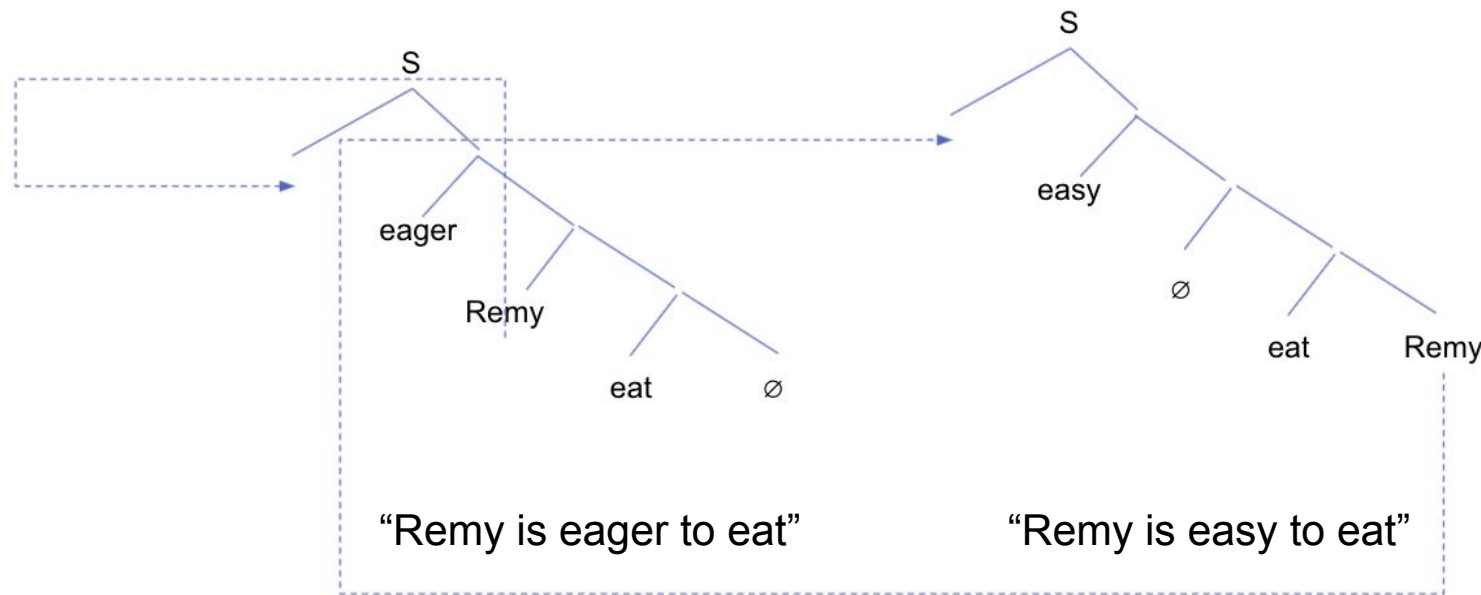
- Explosion of the grammar: {sg, pl} x {masc, fem} = 4 subscripts
- Consider also languages with even more complex inflectional morphology
- Loss of generalization

Agreement Solution

- Solution: Parameterizing the “context-free backbone” (terminals and nonterminals and rules)
- NP[num=X, gender=Y] $\rightarrow$ DET [num=X,gender=Y] N [num=X,gender=Y]
- Examples
 - *El libros* (no consistent way to assign X and Y)
 - *La libro* (no consistent way to assign X and Y)
 - *El libro* (X=SG, Y=MASC)

Beyond CFG: “Movement”

- *Remy* is in the same surface position in the two sentences, but the underlying structures differ
- In one, *Remy* is the underlying **subject** of *eat*; in the other it is the underlying **object**



HMM vs. CFG

HMM

- Start state
- States and transitions
- States emitting symbols
- Path through lattice
- Finite state assumption

CFG

- Start symbol
- Rules
- Pre-terminals, rewriting to terminals
- Derivation tree (down to pre-terminals)
- Context-free assumption

Two Challenges for Finite-state Generation

- “Matching” (The fast fast dog
chased the terrified squirrel)

- CFG solution: **recursion**

Ex: $\{a^n c b^n \mid n \geq 0\}$

CFG:

$$S \rightarrow c$$
$$S \rightarrow a S b$$

* Center-embedded rules, where a non-terminal is rewritten as itself, surrounded by (non-empty) strings

- Generalization
(NP behaving as a unit)

- *Los estudiantes aman el libro.*
- *Aman los estudiantes el libro?*

Handling “Movement” Within CFG

- A. John likes the book
- B. I wrote the book
- c. John likes the book I wrote

- Want to capture “deep” relationship between verb wrote and its object in (C)
- Is there a way to do this *within* CFG, avoiding adding a “movement” operation?

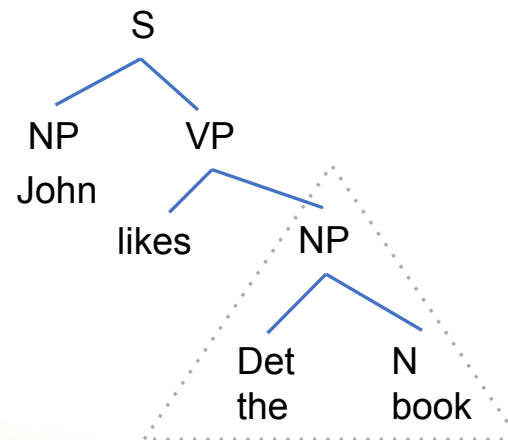
Handling “Movement” Within CFG: Slash Categories

A student: “Slash categories”

S $\square$ NP VP

VP $\square$ V NP

NP $\square$ Det N



NP $\square$ NP S/NP

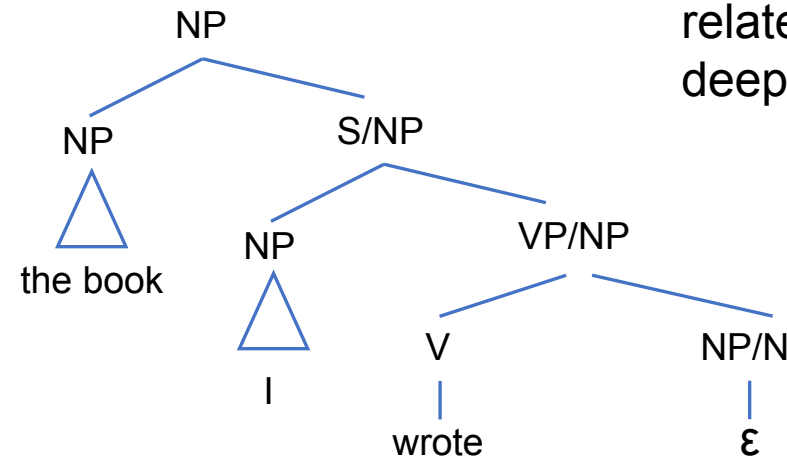
S/NP $\square$ NP VP/NP

VP/NP $\square$ NP/ NP

NP/ NP $\square$ ϵ

trace (empty content)

Note how you can “follow” the /NP to relate surface and deep positions



Combinatory Categorical Grammar Basics

- “**Slash category**” is essentially a nonterminal that is missing something to be complete, e.g. VP/NP (**VP missing an NP to its right**).
Combinatory Categorical Grammar uses a similar idea, without context-free rewrite rules.
- Intuitively, a categorial grammar operates by having a limited set of **atomic** categories (NP, S, PP, ...) but then allowing lexical items to have more complex categories – one 1-argument functions

Eg.

likes := (S\NP) / NP

A transitive verb is a function from NPs into predicate S. That is, it accepts two NPs as arguments and result in S.

<i>Mary</i>	<i>likes</i>	<i>musicals</i>
<i>NP</i>	<i>(S\NP)/NP</i>	<i>NP</i>

Function application

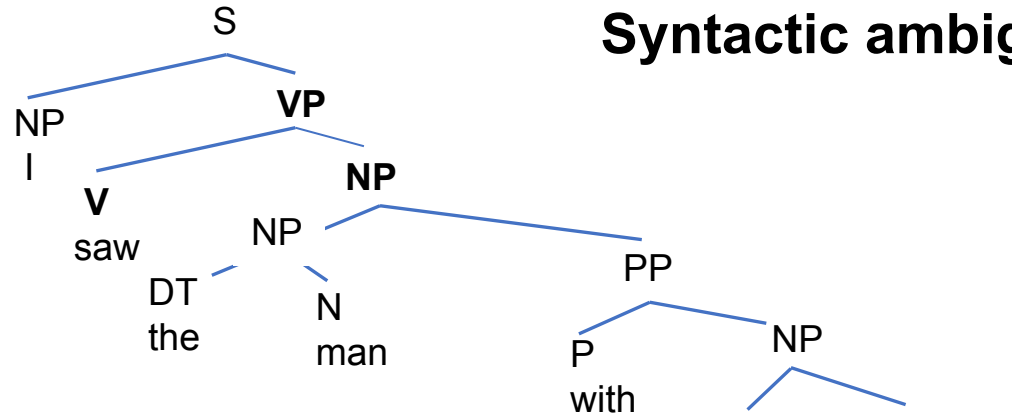
Forward app: $> : X / Y \quad Y \sqcap X$
(S\NP) / NP NP $\sqcap$ NP

Backward app: $< : Y \quad X \backslash Y \sqcap X$
NP S\NP $\sqcap$ S

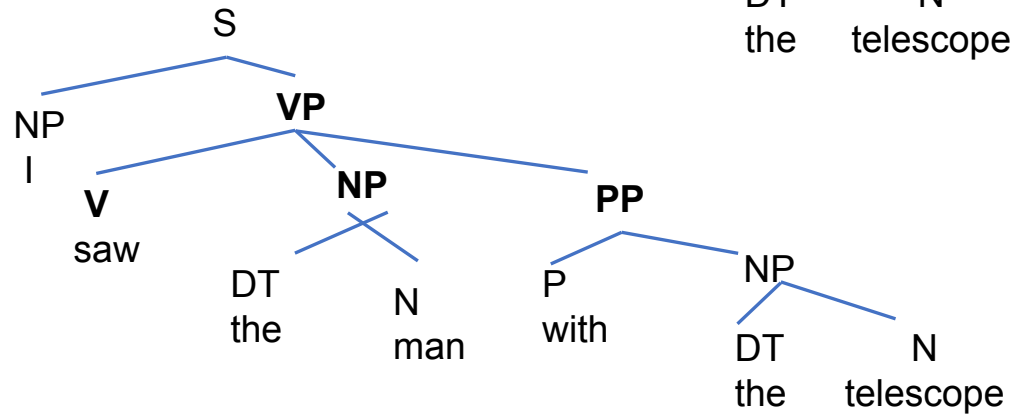
Constituency Parsing

Ex.

VP $\square$ V NP



VP $\square$ V NP PP



- Recall the idea of decoding:
 - Search through a space of unobservable structures, guided by a model
- CFG: Search through possible **trees**

CKY: Dynamic Programming Again!

- Dynamic programming: split the problem to sub-problems and solve them, when complete, the sub-problem solutions solve the problem as whole.
- CKY is applicable to CFGs that are in Chomsky Normal Form
- All rules have the form $X \rightarrow x$ (**preterminal to terminal**) or $X \rightarrow YZ$

Situations we need to address to convert to Chomsky Normal Form:

1. Rules that mix non-terminal and terminal
 $INF-VP \rightarrow to VP \rightsquigarrow INF-VP \rightarrow TO VP$
2. Rules with a single non terminal (unite production)
 $A \rightarrow B \ \& \ B \rightarrow y \text{ (non unite production)} \rightsquigarrow A \rightarrow y$
3. Rules with right hand-side longer than 2
 $A \rightarrow BCy \rightsquigarrow X1 \rightarrow BC \ \& \ A \rightarrow X1y$

Illustration of conversion to Chomsky Normal Form

CKY

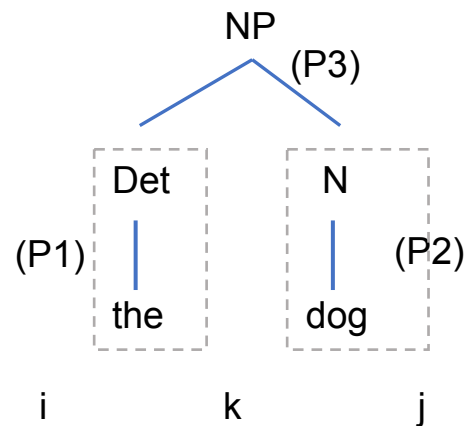
- **Recognition:** Is there an S in cell $[0, n]$?
 - **Parsing:** Record all ways to arrive at S in $[0, n]$ given the input
- CKY was first $O(n^3)$ algorithm to parse CFGs, but limited to CNF
- Earley then introduced the first $O(n^3)$ algorithm for **arbitrary** CFGs

S $\rightarrow$ NP VP
VP $\rightarrow$ VP PP
VP $\rightarrow$ V NP
VP $\rightarrow$ eats
PP $\rightarrow$ P NP
NP $\rightarrow$ Det N
NP $\rightarrow$ she
V $\rightarrow$ eats
P $\rightarrow$ with
N $\rightarrow$ fish
N $\rightarrow$ fork
Det $\rightarrow$ a

6	S						
5		VP					
4							
3	S						
2		VP			PP		
1	S		NP			NP	
0	NP	V, VP	Det.	N	P	Det	N
	she	eats	a	fish	with	a	fork
	0	1	2	3	4	5	6

Probabilistic CKY

- What about the goal of finding the **best** parse, if there's ambiguity?
- Use **probabilistic CFG (PCFG)**, which is a CFG where rules are augmented with probabilities
- Model parameters: $P(A \rightarrow r \mid A)$ such that $\sum_y P(A \rightarrow r \mid A) = 1$



Probability of *this* NP spanning from i to j is the product of three probabilities:

P1 (probability of having a Det spanning from i to k)

P2 (probability of having an N spanning from k to j)

P3 (probability of rule $NP \rightarrow Det\ N$ given that we are expanding NP)

Probability of *any* NP spanning from i to j is the *sum* of such probabilities, for *all* the different ways you could build an NP spanning from i to j

Viterbi CKY parsing is directly analogous to Viterbi decoding for HMMs: if you want the *most probable* latent structure, simply replace **sum** with **max** in the dynamic programming algorithm.

Training (Estimating Parameters) for PCFGs

Recall, supervised training for HMMs if you have an annotated corpus:

DT	ADJ	N	V	DT	ADJ	N
the	brown	dog	chased	a	hungry	squirrel

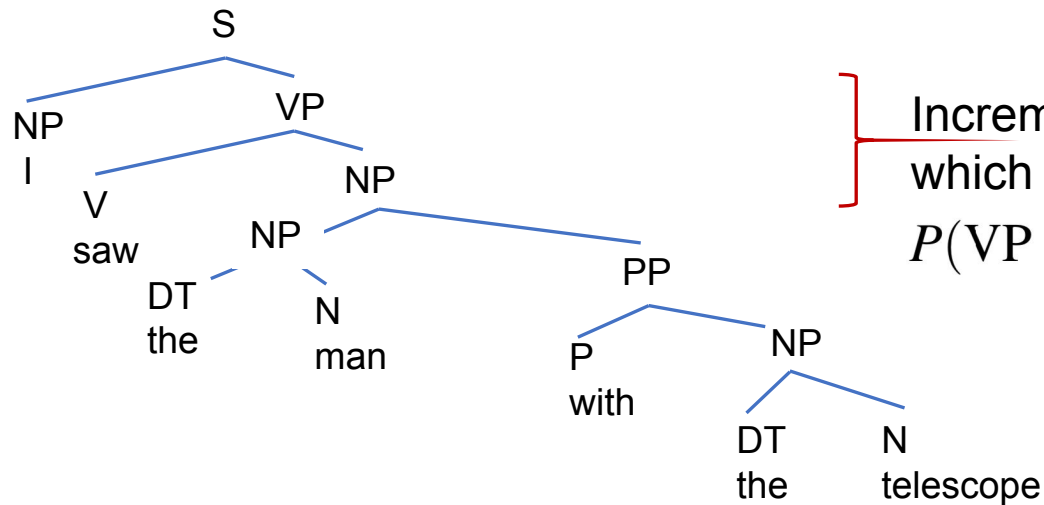
count (t_i, t_j) ----- ☐ A matrix

count (t_i, w_i) ----- ☐ B matrix


smoothing

Training for PCFGs

For CFGs, supervised training is similar using a **treebank** (**parsed corpus**)



Increment count of VP expanding to V NP,
which increases the estimated probability of
 $P(\text{VP} \rightarrow \text{V NP} \mid \text{V})$

What About Human Parsing?

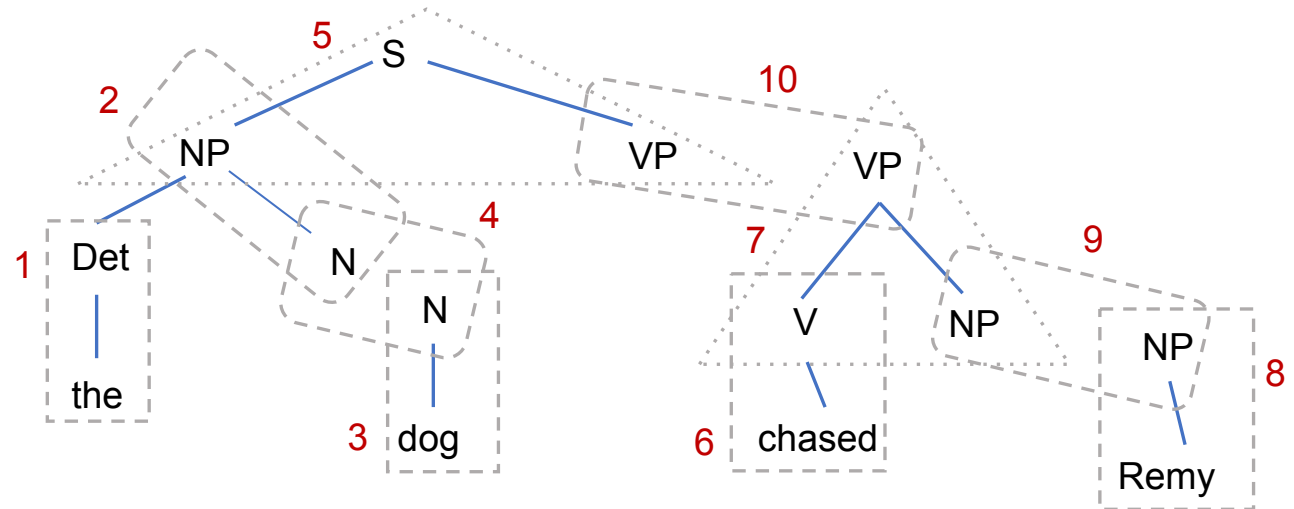
- Sometimes, we don't want to wait to the end of a sentence to start analyzing it
 - Human parsing is incremental
 - Sometimes, we can commit to an incorrect parsing choice because of local ambiguity
 - Occurs with “garden path” sentences, e.g. The old man the boat.

Left-corner Parsing

- There is good reason to believe that humans parse language using some form of **left-corner parsing strategy**

$A \rightarrow r_1 r_2 \dots r_k$

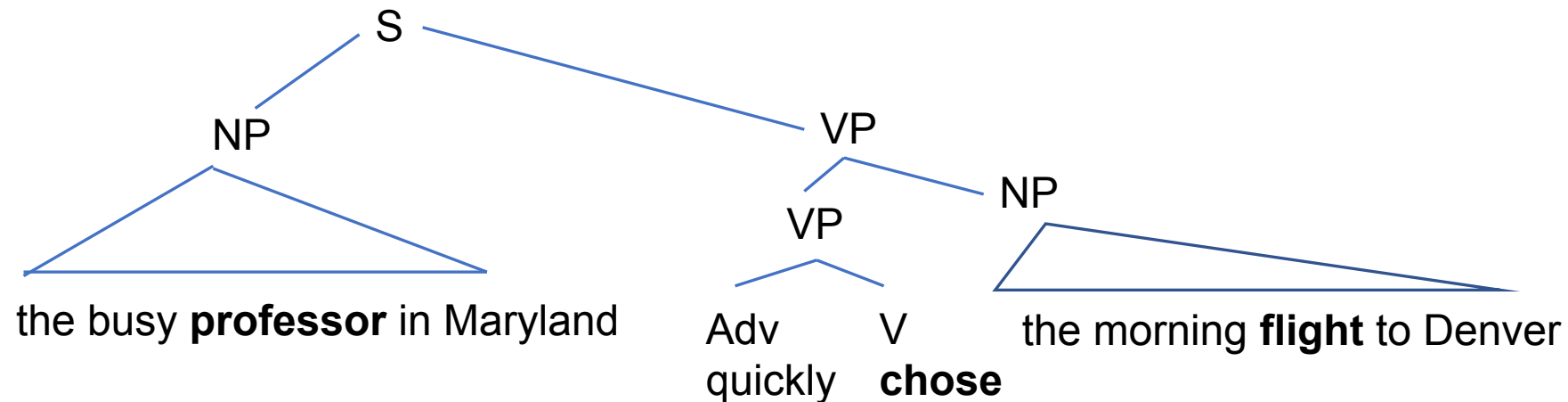
“Left corner” of the context-free rule



- Left-corner identified bottom-up leads to top-down prediction of the rest of the rule
- Example: Once we have seen **Det**, rule $NP \rightarrow Det N$ leads us to **predict** an N

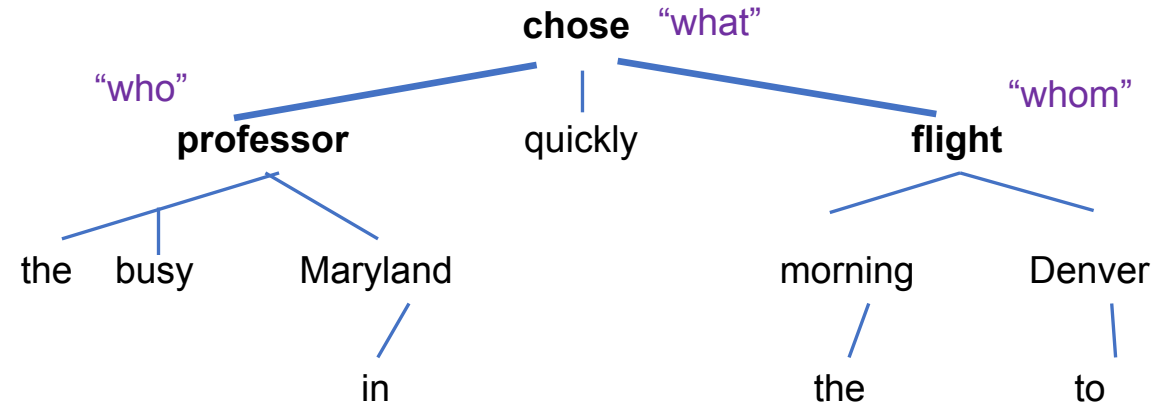
Syntactic Dependency Representations (1/2)

In a constituency tree, the “who did what to whom” relationships are not explicit:

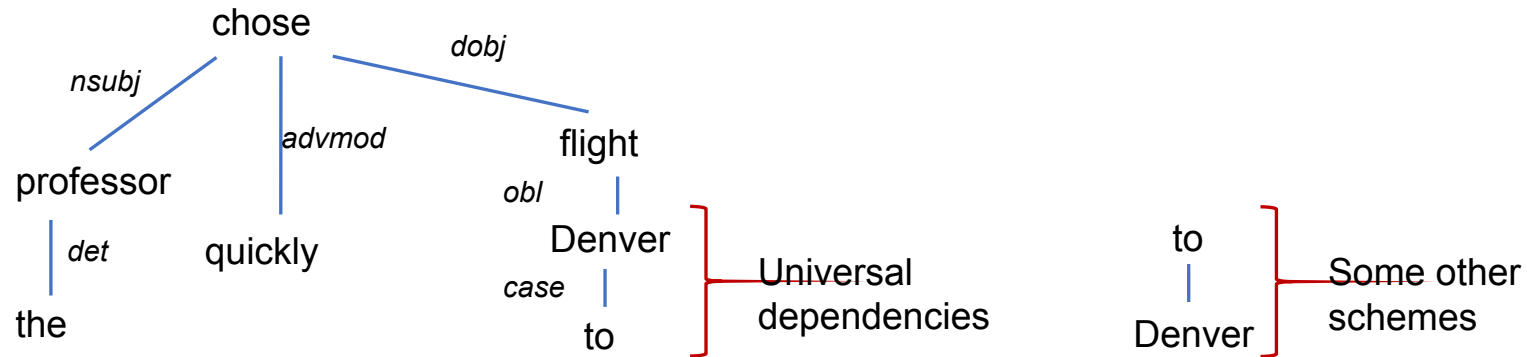


Syntactic Dependency Representations (2/2)

Syntactic dependency representations focus directly on the relations between **heads** and **dependents** (a.k.a. **modifiers**)



Labeled Dependencies



“Flat” notation:

